Discovery Executive Summary

Project: discovery12345 · **Generated:** 7/17/2026, 2:01:46 PM

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 6 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—
2	Code Quality & Complexity Analysis	HIGH RISK	76 / 100 — High Risk
3	Frontend Modernization Analysis	HIGH RISK	—
4	Backend Modernization Analysis	HIGH RISK	—
5	Testing & Quality Assurance Analysis	HIGH RISK	—
6	Security Analysis	HIGH RISK	—

---

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk cross-domain coupling (H8), legacy React Router v5 patterns (F5), and duplicated domain logic in view components (H10).

EXECUTIVE SUMMARY

This TARGET_WORKSPACE is a **frontend-only** social-network SPA (`social-media-react`) with **90 source files** (12 pages, 45 components, 11 services, 4 Redux modules). **No backend/server layer exists in the repository** — all persistence goes through an external REST API at `localhost:3030/api/` (production: `/api/`). Architectural health is **mixed**: HTTP access is partially centralized via `httpService` and domain service modules, but **presentation components bypass Redux** in 16 files, **business rules are duplicated** across post/comment/reply components, and **four feature domains** (user, post/comment, chat, activity) are tightly coupled through shared Redux state, socket orchestration in

`Main.jsx`, and direct cross-service calls. The dominant risks are **legacy React Router v5 patterns (20 files)**, **cross-domain coupling (8 access points)**, and **inconsistent data-flow conventions** that will amplify change cost as features grow.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	N/A (no backend)	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	0 backend handlers	GOOD
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	0 (external API)	GOOD
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0 cycles (88 files, 137 edges)	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	0 (`utilService` is generic)	GOOD
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	N/A (no backend)	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (max: `postActions.js` 210 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	N/A (schema external)	GOOD
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	78.6 avg (58 `.jsx` files)	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	16	MODERATE
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (max: `Message.jsx` 250 LOC)	GOOD
F4	Prop Drilling / Global State	Max prop-drilling depth	≤2	3–4	>4	2 levels; 71 `useSelector`	MODERATE

	Abuse					hooks	
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	20	HIGH RISK
H10	Duplicated Domain Logic in Views (additional)	Components duplicating reaction/connection rules	0	1–3	>3	5	MODERATE

1.4 Actions Required

Hotspot	Action	Rating	Priority
H8	Split cross-domain imports: move comment actions out of `postActions.js`, add `activityFacade` for notification DTO assembly, decentralize socket subscriptions from `Main.jsx`	HIGH RISK	CRITICAL
F5	Migrate React Router v5 (`HashRouter` , `component=` , `useHistory`) to v6 across 20 files; standardize service naming	HIGH RISK	HIGH
F2	Eliminate 16 direct service imports from components — route all reads through Redux thunks and selectors	MODERATE	HIGH
F4	Introduce `AuthContext` + memoized selectors; document single data-access convention	MODERATE	MEDIUM
H10	Extract shared `toggleReaction` and `addConnection` domain helpers; remove duplicated logic from 5 view files	MODERATE	MEDIUM

1.5 Expected Outcomes

- **Bounded feature modules** — user, post, comment, chat, and activity can evolve independently with explicit facades instead of cross-imports.
- **Single data-flow path** — components consume selectors/thunks only, eliminating dual Redux + direct-service paths and stale cache bugs in `userService.usersCash` .
- **Testable domain rules** — extracted reaction/connection helpers become unit-testable without mounting React components.
- **Lower migration cost** — React Router v6 upgrade unlocks modern layouts, error boundaries, and code-splitting patterns.
- **Immutable, predictable state** — normalized Redux store removes embedded comment graphs and direct prop mutation anti-patterns.

---

2. Code Quality & Complexity Analysis

OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY

High Risk

Driven by H1 High Cyclomatic Complexity (max CC 41) and H5 Duplicate Code (~12.7%).

EXECUTIVE SUMMARY

This analysis covered **96 frontend source files** across two React applications (`social-media-react` at 87 files, `workbench-demo` at 9 files). No backend/server application layer exists in TARGET_WORKSPACE. Overall code quality is **High Risk**, driven by **high cyclomatic complexity** in notification and messaging components (max CC **41**) and **general duplicate code** estimated at **~12.7%** of normalized frontend LOC. File and class sizes remain within good bounds (largest file **212 LOC**; no files exceed 1000 LOC). Redux action modules repeat identical async thunk boilerplate across four files (**42 instances**). Git churn, defect-prone file, and ownership metrics could not be computed because target application sources are **untracked** in the parent repository history.

2.1 Benchmark Ratings Summary

Manual cyclomatic inspection (branch/loop/ `&&` / `||` / `?:` counting) was used; no ESLint `complexity` rule or Sonar config is present in either project. Duplicate-code percentage derived from normalized 5-line block matching across `social-media-react/src` and `workbench-demo/src`.

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	41 (NotificaitonPreview)	HIGH RISK
H2	Large Classes	Largest class LOC	<300	300–1000	>1000	212 (LoginPage.test.tsx / Message.jsx)	GOOD
H3	Large Functions	Largest function LOC	<50	50–200	>200	190 (Signup.jsx `Signup`)	MODERATE
H4	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	~8% (Redux thunk + auth-form patterns)	MODERATE
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	~12.7% (5-line block analysis)	HIGH RISK
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	n/a (source untracked in git)	GOOD
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	n/a (source untracked in git)	GOOD
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	n/a (source untracked in git)	GOOD

H9	Fragmented useEffect Chains (additional)	Max useEffect count per component (target ≤2)	≤2	3	≥4	3 (NotificaitonPreview.jsx)	MODERATE

No additional hotspots beyond H9 were observed.

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25% → 50%	88	44.0
Code Churn	25%	n/a	n/a
Defect Density	20%	n/a	n/a
Class/Function Size	15% → 30%	55	16.5
Business Logic Duplication	10% → 20%	75	15.0
Developer Ownership Risk	5%	n/a	n/a
Hotspot Score	**100%**		**76 / 100**

2.5 Actions Required

Hotspot	Action	Rating	Priority
H1 High Cyclomatic Complexity	Extract `ActivityMessageStrategy` for notification types; split `useChat` hook in `Message.jsx` ; enable ESLint `complexity` rule (threshold 15)	HIGH RISK	CRITICAL
H3 Large Functions	Extract `AuthFormFields` from `Home.jsx` / `Signup.jsx` ; decompose `CommentPreview` handlers into service helpers	MODERATE	MEDIUM
H4 Business Logic Duplication	Introduce shared Redux async action factory; consolidate activity copy into `buildActivityMessage()`	MODERATE	MEDIUM
H5 Duplicate Code (general)	Create shared `FormInputGroup` component; add `test-utils.tsx` helpers in workbench-demo	HIGH RISK	HIGH
H9 Fragmented useEffect Chains	Replace 3-effect chains in `NotificaitonPreview.jsx` with `useNotificationPreview` hook or React Query	MODERATE	MEDIUM

2.6 Expected Outcomes

- Lower defect rate in notifications and messaging flows by isolating activity-type logic behind Strategy handlers (CC target <15 per function).
- Faster code reviews and safer refactors as shared form components and Redux thunk factories eliminate ~8–13% duplicated LOC.

- Improved testability via smaller extracted hooks (`useChat*` , `useNotificationPreview`) that can be unit-tested independently of page components.
- Reduced stale-closure and double-fetch bugs from consolidated effect management in notification and search components.
- Future discovery runs can track churn and ownership once frontend projects are committed to git, enabling proactive maintenance of high-change files.

Full report saved to `target/docs/discovery/02-code-quality-complexity.md` (321 lines). Pipeline summary: `agent-runs/20260717T132536_tkr1f1/02-code-quality-complexity-summary.md` .

---

3. Frontend Modernization Analysis

OVERALL CODEBASE RATING — FRONTEND MODERNIZATION

High Risk

Driven by H1 UI component duplication (~15% of scanned files replicate loading/preview patterns without a shared library) and H6 legacy Redux store architecture (0% RTK adoption).

EXECUTIVE SUMMARY

The `target/` workspace contains two React single-page applications. The primary surface is `social-media-react` (React 18.2.0, legacy Redux `createStore` + thunks, React Router v5), comprising 58 JSX view units plus 3 TypeScript components in `workbench-demo` (React 18.3.1, hooks-only, Tailwind). All 61 scanned components are functional — no class-based components were found. The most severe modernization gaps are duplicated UI patterns (inline loading spinners in 9 files, repeated like-toggle logic in 4 preview components) and legacy global Redux wiring read by 49% of components. `Message.jsx` (250 LOC) mixes chat orchestration, socket side-effects, and view composition in one file, and the messaging feature drills 10+ props through four component layers without a domain composable or context.

3.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	~15% (9/61 files)	HIGH RISK
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	100% (61/61)	GOOD
H3	Massive Components	Largest component LOC	<200	200–500	>500	250 (`Message.jsx`)	MODERATE

H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	49% (30/61)	MODERATE
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	4 levels (Message → Messaging → ListMsg → MsgPreview)	MODERATE
H6	Legacy Redux Store (additional)	RTK/modern store adoption %	>90%	70–90%	<70%	0% (0/4 modules use RTK)	HIGH RISK
H7	Direct DOM in React Views (additional)	Components using imperative DOM APIs	0 files	1–2 files	>2 files	2 files	MODERATE

3.5 Actions Required

Hotspot	Action	Rating	Priority
H1 UI Component Duplication	Create `src/cmps/shared/` with `LoadingSpinner`, `UserAvatarCard`, and `useLikeReaction`; replace 9 loading blocks and 4 duplicate like handlers	HIGH RISK	HIGH
H3 Massive Components	Extract `useChat` hook from `Message.jsx`; split `CommentPreview.jsx` and `CreatePostModal.jsx` into subcomponents under 200 LOC each	MODERATE	MEDIUM
H4 Global State Dependencies	Migrate `social-media-react/src/store/` to Redux Toolkit slices; add memoized selectors to reduce ad-hoc `useSelector` inline lambdas	MODERATE	HIGH
H5 Complex State Management	Introduce `ChatContext` / `useMessagingState` composable to eliminate 4-level prop drilling in messaging and comment save chains	MODERATE	MEDIUM
H6 Legacy Redux Store	Add `@reduxjs/toolkit`, convert four reducer modules to slices, adopt RTK Query for `userService` / `postService`	HIGH RISK	CRITICAL
H7 Direct DOM in React	Refactor `InputFilter.jsx` to controlled autocomplete list; replace `MessageThread.jsx` scroll with `useRef`	MODERATE	MEDIUM

3.6 Expected Outcomes

- A shared component library (`LoadingSpinner`, `UserAvatarCard`, `ReactionButton`) reduces duplicated markup from ~15% of files to near zero and enforces consistent loading and interaction UX.
- Extracting `useChat`, `useLikeReaction`, and RTK slices improves unit-test coverage for chat and reaction flows without mounting full page trees.
- Redux Toolkit migration eliminates ~872 LOC of manual boilerplate and provides typed selectors, reducing the 49% global-store coupling through scoped domain hooks.
- `ChatContext` collapses 10-prop drilling chains in messaging to single-hook consumption, simplifying addition of typing indicators and read receipts.

- Replacing imperative DOM calls in `InputFilter` and `MessageThread` restores React-idiomatic rendering and prevents event-listener leaks during hot reload.

---

4. Backend Modernization Analysis

OVERALL CODEBASE RATING — BACKEND MODERNIZATION

High Risk

Driven by H6 (0% documented/governed REST surface) and H7 (0% API governance compliance: no OpenAPI, versioning, or contract tests).

EXECUTIVE SUMMARY

The **ERM Complexity Demo** backend is a compact Spring Boot 2.7.18 monolith (17 Java source files, <5K LOC) with a deliberate legacy-debt surface: static shared-business facades, duplicated controller logic, and dual Oracle/PostgreSQL dialect routing. Layering is partially sound—JPA access is confined to `RiskService` and `DemoDataLoader`, and controllers inject services rather than repositories—but modernization gaps remain in API governance and static coupling. Eight read-only REST endpoints under `/api` serve AngularJS panels with **no OpenAPI spec, no versioning, and no contract tests** (0% governance compliance). Dynamic-variable-from-input patterns were not observed; however, `SharedBusinessServices` static methods, a Spring singleton holding mutable RBAC state, N+1 repository calls in domain aggregation, and plaintext database passwords in profile properties require remediation before production hardening.

4.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Dynamic Variable Creation	Dynamic-var-from-input occurrences	0	1–10	>10	0	GOOD
H2	Global Mutable State	Globals / mutable static state	0	1–5	>5	1	MODERATE
H3	Direct SQL Outside Data Layer	Data-layer compliance %	>90%	60–90%	<60%	100%	GOOD
H4	Static / Singleton Abuse	Business-logic static/singleton classes	0	1–5	>5	1	MODERATE
H5	Missing Service Layer	Handlers with inline business logic	<10	10–20	>20	2	GOOD

H6	API Sprawl	Documented & governed endpoints %	>90%	80–90%	<80%	0%	HIGH RISK
H7	Missing API Governance	Governance compliance %	100%	90–99%	<90%	0%	HIGH RISK
H8	N+1 Repository Calls (additional)	Service methods with per-item repository loops	0	1–2	>2	1	MODERATE
H9	Hardcoded Credentials (additional)	Plaintext secrets in committed config files	0	1–5	>5	2	MODERATE

4.5 Actions Required

Hotspot	Action	Rating	Priority
H6 API Sprawl	Consolidate risk reads under versioned `/api/v1/risks`; deprecate duplicate MVC JSON paths; publish OpenAPI listing all 8 endpoints	HIGH RISK	HIGH
H7 Missing API Governance	Add springdoc-openapi, `/api/v1` prefix on `ApiController`, and MockMvc contract tests for `/health` and `/risks` response shapes	HIGH RISK	CRITICAL
H2 Global Mutable State	Externalize `AccessControlService` RBAC matrix to immutable `@ConfigurationProperties` or policy service; inject via `PermissionPolicyPort`	MODERATE	MEDIUM
H4 Static / Singleton Abuse	Replace `SharedBusinessServices` static methods with injectable `TenantDialectService` and `RiskIdComposer` beans	MODERATE	MEDIUM
H8 N+1 Repository Calls	Refactor `RiskService.countsByDomain()` to a single `@Query` GROUP BY or `countByDomain` repository method	MODERATE	MEDIUM
H9 Hardcoded Credentials	Move Postgres/Oracle passwords from `application-*.properties` to environment variables or Spring Cloud Config	MODERATE	HIGH

4.6 Expected Outcomes

- OpenAPI-backed `/api/v1` endpoints give AngularJS and future consumers a stable, versioned contract and prevent silent breaking changes to `RiskItem` JSON shapes.
- Extracting `SharedBusinessServices` into injectable services enables per-region dialect routing via Spring profiles and unit-test isolation without static coupling.
- Externalizing RBAC policy and database credentials removes mutable singleton state and plaintext secrets from committed configuration.
- A single `RiskService.findByDomainCode()` eliminates duplicated controller logic and reduces API sprawl between MVC and REST entry points.
- Replacing N+1 domain count queries with aggregated repository methods improves dashboard load time as the risk register scales beyond demo seed data.

Full report saved to `target/docs/discovery/04-backend-modernization.md` (406 lines). Verified against **Java 8 / Spring Boot 2.7.18** backend in `glmarvel29-ai/global-hr-modernization`: 17 Java files, 2 controllers, 8 REST endpoints, 0 test files, 0% API governance.

---

5. Testing & Quality Assurance Analysis

OVERALL CODEBASE RATING — TESTING & QUALITY ASSURANCE

High Risk

Driven by zero test coverage (H2), seven untested critical modules (H1), no integration or contract tests (H3/H4), absent CI gate (H6), and no frontend/E2E or Maven test tooling (H7/H8).

EXECUTIVE SUMMARY

The `global-hr-modernization` repository is a Java 8 / Spring Boot 2.7 WAR application with a JSP + AngularJS 1.8 frontend layer. Analysis via the GitHub REST API on branch `main` found **zero test files** (`src/test` absent), **no test dependencies** in `pom.xml` (no `spring-boot-starter-test`, JaCoCo, or Surefire coverage gates), and **no CI workflow** (`.github/workflows` not present). Estimated coverage is **0% backend** (17 Java source files) and **0% frontend** (1 AngularJS module + 5 JSP views + 1 CSS file). Seven business-critical modules—including `AccessControlService` (RBAC/ABAC), `RiskService`, `ApiController`, and `DemoDataLoader`—ship with no automated tests. The README quick-start explicitly runs `mvn package -DskipTests`, reinforcing that tests are not part of the delivery path. Overall testing posture is **High Risk** and must be addressed before any modernization or extraction work.

5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests	0	1–3	>3	7 critical modules, 0 tests	HIGH RISK
H2	Low Test Coverage	Overall coverage %	>80%	50–80%	<50%	0% backend · 0% frontend (test-file ratio)	HIGH RISK
H3	Missing Integration Tests	Boundaries covered %	>70%	30–70%	<30%	0% (0 of ~5 key boundaries)	HIGH RISK
H4	Missing Contract Tests	APIs with contract tests %	>80%	40–80%	<40%	0% (0 of 8 REST endpoints)	HIGH RISK

H5	Flaky / Skipped Tests	Skipped/flaky test count	0	1–5	>5	0 skipped/disabled tests	GOOD
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	No `.github/workflows`; no CI detected	HIGH RISK
H7	No End-to-End / Frontend Tests (additional)	UI flows covered by E2E or component tests (%)	>60%	20–60%	<20%	0% (0 specs for 5 JSP pages + AngularJS shell)	HIGH RISK
H8	No Maven Test Tooling / Coverage Gate (additional)	Build enforces test deps + coverage threshold	Yes	Partial (tests run, no threshold)	No test deps or gate	`pom.xml` lacks `spring-boot-starter-test`, JaCoCo, Surefire gate	HIGH RISK

5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 Untested Critical Logic	Add JUnit 5 unit tests for `AccessControlService`, `RiskService`, `SharedBusinessServices`, and slice tests for `ApiController` / `PageController`	HIGH RISK	CRITICAL
H2 Low Test Coverage	Add `spring-boot-starter-test`, create `src/test/java` mirroring main packages; target 75% JaCoCo line coverage before refactor	HIGH RISK	CRITICAL
H3 Missing Integration Tests	Add `@DataJpaTest` for repository, `@SpringBootTest` for `DemoDataLoader` seeding, MockMvc for controller wiring	HIGH RISK	HIGH
H4 Missing Contract Tests	Add MockMvc/REST Assured tests for all 8 `/api/*` endpoints asserting status, content-type, and required JSON fields	HIGH RISK	HIGH
H6 No CI Test Gate	Create `.github/workflows/ci.yml` running `mvn verify` on push/PR with JDK 8+17 matrix; require as branch protection check	HIGH RISK	CRITICAL
H7 No E2E / Frontend Tests	Add Playwright E2E specs for 5 JSP routes and Karma unit tests for `erm-app.js` AngularJS controllers	HIGH RISK	HIGH
H8 No Maven Test Tooling	Add test dependencies, Surefire, and JaCoCo plugins to `pom.xml`; create missing `docs/START_TEST_AND_ISSUES.md`	HIGH RISK	MEDIUM

5.5 Expected Outcomes

- RBAC/ABAC security logic and risk register queries are protected by unit tests before any service extraction or modernization.
- Integration tests validate JPA persistence, startup seeding, and dual-database dialect routing across H2/PostgreSQL/Oracle profiles.

- Contract tests on eight REST endpoints prevent breaking JSON shape changes from reaching the AngularJS frontend undetected.
- CI runs `mvn verify` on every pull request, blocking merges when tests fail or coverage drops below the JaCoCo threshold.
- Playwright E2E tests cover the five JSP dashboard pages, enabling safe JSP-to-SPA migration with a regression safety net.

---

6. Security Analysis

OVERALL CODEBASE RATING — SECURITY

High Risk

Driven by DOM XSS, client-side-only auth bypass, secrets in the bundle, and 61 critical/high npm audit findings.

EXECUTIVE SUMMARY

The target workspace contains two React 18 single-page applications — **social-media-react** (Redux, React Router v5, Socket.io, session-cookie API client) and **workbench-demo** (TypeScript login demo, React Router v6) — with **no server-side application code** present locally; security depends on an external API referenced at `localhost:3030`. Review covered **both frontend layers** and dependency manifests. The most severe findings are **DOM-based XSS** via unsanitized `innerHTML` in search autocomplete, **client-side-only route protection** (including an unguarded `/dashboard` in workbench-demo), **JWT and user objects stored in browser storage**, a **hard-coded Google Maps API key** shipped in the client bundle, and **115 npm audit findings** (8 critical, 53 high across both apps). No CSP or security headers were found in either `public/index.html`. Overall posture is **High Risk**, driven by exploitable client-side access-control bypass, XSS, exposed secrets, and vulnerable/outdated dependencies.

6.1 Security Benchmark Ratings

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	1	MODERATE
H2	High Vulnerabilities	0	<5	5–10	>10	12	HIGH RISK
H3	Medium Vulnerabilities	low	<20	20–50	>50	4	GOOD
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	2.5/KLOC	HIGH RISK
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	17% clean (2/12)	HIGH RISK

H6	Critical/High Vulnerable Deps	0	0	1	>1	61	HIGH RISK
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	~35% direct deps outdated/EOL	HIGH RISK
H8	End-of-Life Dependencies	0	0	1–5	>5	2 (axios 0.27.x, react-router-dom 5.x)	MODERATE

6.5 Actions Required

Finding	Action	Rating	Priority
DOM XSS via innerHTML autocomplete	Replace `innerHTML` in `InputFilter.jsx` with React-rendered list; encode user display names server-side	HIGH RISK	HIGH
Client-side-only route protection	Add server-validated `ProtectedRoute` in both apps; guard `/dashboard` in workbench-demo	HIGH RISK	HIGH
JWT in localStorage	Remove token from `localStorage`; use HttpOnly session cookies	HIGH RISK	HIGH
User object in sessionStorage	Stop trusting `sessionStorage` user; bootstrap via `GET auth/me`	HIGH RISK	HIGH
Hard-coded Google Maps API key	Rotate key; move to env var with referrer restrictions	HIGH RISK	HIGH
Vulnerable npm dependencies (115 total)	Upgrade axios, react-router-dom; run `npm audit fix`; add CI gate	HIGH RISK	CRITICAL
Missing CSP / security headers	Add CSP to both `public/index.html` or reverse-proxy headers	MODERATE	MEDIUM
Unvalidated user URLs in posts	Allow-list http/https URLs for `link`, `imgUrl`, `videoUrl`	MODERATE	MEDIUM
No DevSecOps scanning in CI	Add `npm audit --audit-level=high` and secret scanning to pipeline	HIGH RISK	HIGH
No security audit logging	Log auth failures and access denials server-side; avoid logging tokens	MODERATE	MEDIUM